

ECN-207 Questions from 2022

- (1 point) In serial execution, we run 1000 tasks on one processor. In parallel execution, we run 10000 tasks on ten processors. Which of these can we apply here? (a) Weak-scaling (b) Strong-scaling
(a)
- (4 points) We execute 500 instructions in 45 cycles, 600 instructions in 55 cycles and 700 instructions in 35 cycles. Show calculation of weighted harmonic and arithmetic mean. Clearly show weights, such that the sum of weights should be equal to 1.

If someone computed IPC:

For WAM, Weights are 45/135, 55/135, 35/135, respectively.

$$\text{WAM} = (45/135) * (500/45) + (55/135) * (600/55) + (35/135) * (700/35) = 13.33$$

For WHM, Weights are 500/1800, 600/1800, 700/1800, respectively.

$$(\text{WAM}) = 1/[(500/1800) * (45/500) + (600/1800) * (55/600) + (700/1800) * (35/700)] = 13.33$$

If someone computed CPI [They must have clearly written that they are computing CPI:

For WAM, weights are 500/1800, 600/1800, 700/1800, respectively.

$$(\text{WAM}) = (500/1800) * (45/500) + (600/1800) * (55/600) + (700/1800) * (35/700) = 0.075$$

For WHM, weights are 45/135, 55/135, 35/135, respectively.

$$\text{WAM} = 1/[(45/135) * (500/45) + (55/135) * (600/55) + (35/135) * (700/35)] = 0.075$$

- (2 points) Consider a FP (floating point) number system with total T bits and E exponent bits and 1 sign bit. How many total representations for NaN will be there in this number system if T=12 and E=6.
Fraction bits = 5.

Total positive NaN = $2^5 - 1$. Total NaNs are double of this, which is 62. So, answer is 62.

Partial marking: If someone wrote only 31, they get 1 mark.

- (5 points) Assume we use a new-precision floating point number system. It has a total of 17 (seventeen) bits, of which 1 bit is for sign, 6 bits for exponent and remaining bits for fraction. It differs from IEEE-754 such that it uses a bias value of 15 (instead of that derived from IEEE-754 formula).
(i) Find out the smallest and largest 'normal' positive numbers which can be represented in new-precision format (i.e., ignore denormal numbers and negative numbers). You should leave the answer in base 2 format. Do not approximate the value of smallest number. (3 marks)

For smallest number:

Exponent: 000001. Bias is 15, so actual exponent is $1-15 = -14$

Significand is 1, so actual value of smallest number is $1 * 2^{-14}$ [1.5 marks]

For largest number:

Exponent: 111110, which is 62. Bias is 15, so actual exponent is $62-15 = 47$

Significand is nearly 2, so the value of largest number is $2 * 2^{47}$. This solution also gets 1.5 marks (if someone has approximated it).

Without approximating, largest number is $1.1111111 * 2^{47}$ [1.5 marks]

Partial marking: If someone assumed the bias incorrectly to be 31 (as per IEEE-754 formula), then following result will be obtained. [No other bias value will lead to any marks.]

For smallest number:

Exponent: 000001. Bias is 31, so actual exponent is $1-31 = -30$

Significand is 1, so actual value of smallest number is $1 * 2^{-30}$ [1 marks]

For largest number:

Exponent: 111110, which is 62. Bias is 31, so actual exponent is $62 - 31 = 31$

Significand is nearly 2, so the value of largest number is $2 * 2^{31}$. This solution also gets 1 marks (if someone has approximated it).

Without approximating, largest number is $1.1111111 * 2^{31}$ [1 marks]

(ii) Now, find the smallest and largest positive denormal numbers in new-precision format. (2 marks)

For denormal numbers, exponent is 2^{-14} .

Smallest denormal number [1 mark]:

$$0.00..01 * 2^{-14} = 2^{-10} * 2^{-14} = 2^{-24}$$

Largest denormal number [1 mark]:

$$0.11...11 * 2^{-14} = (1 - 2^{-10}) * 2^{-14} = 2^{-14} - 2^{-24}$$

Partial marking: If someone assumed the bias incorrectly to be 31 (as per IEEE-754 formula), then following result will be obtained. [No other bias value will lead to any marks.]

For denormal numbers, exponent is 2^{-30} .

Smallest denormal number [0.75 mark]:

$$0.00..01 * 2^{-30} = 2^{-10} * 2^{-30} = 2^{-40}$$

Largest denormal number [0.75 mark]:

$$0.11...11 * 2^{-30} = (1 - 2^{-10}) * 2^{-30} = 2^{-30} - 2^{-40}$$

For partial marking, if answers are different from above shown, no marks will be given.

5. (5 points) An application has 3 types of instructions: 25% LoadStore, 40% Arithmetic and Control (35%). LoadStore takes 3 cycle, Arithmetic takes 4 cycle and control takes 2 cycle. Processor frequency = 500MHz.

(a) [0.5 marks] Find CPI.

$$\text{CPI} = .25*3 + .4*4 + .35*2 = 3.05$$

(b) [1 marks] CPU takes 13 seconds to execute this application. Find its speed in MIPS (million instructions per second).

$$\text{MIPS} = \text{clockRate} / (\text{CPI} * 10^6) = 500\text{MHz} / (3.05 * 10^6) = 500 / 3.05 = 163.9$$

(c) [0.5 marks] We apply Optimization1 whereby cycle time is increased by 25%. Find new frequency.

$$\text{New frequency} = 500 / 1.25 = 400 \text{ MHz}$$

(d) [1 marks] We apply Optimization2 whereby number of Arithmetic instructions is reduced by half. Find new CPI

$$\text{New CPI} = (.25*3 + .2*4 + .35*2) / 0.8 = 2.81$$

(e) [2 marks] Find how many CPU seconds will program take if both Optimization2 and Optimization1 are applied together.

$$\text{CPU seconds} = (\text{Number of instructions} * \text{Number of Clocks per instructions}) / \text{Clock Rate}$$

$$\text{Originally, we executed } (163.9 * 10^6) * 13 = 2130.7 * 10^6 \text{ instructions in 13 seconds}$$

Of these, 40% were Arithmetic, i.e. total of $852.2 * 10^6$ instructions.

After Optimization2, there are $426.14 * 10^6$ Arithmetic instructions and a total of $1704.56 * 10^6$ instructions.

$$\text{New time} = \text{instructions} * \text{CPI} / \text{Frequency} = (1704.56) * (2.81 / (400)) = 11.97 \text{ seconds}$$

6. (1 point) For the following C statement, write the corresponding RISC-V assembly code. Assume that the variables f, g, h, i, and j are assigned to registers x5, x6, x7, x28, and x29, respectively.

```
if(i > j)
    f = g-h
```

Here is one possible answer. If someone has written a NOP at EXIT, it is OK:

```
blt x28, x29, EXIT
beq x28, x29, EXIT
sub x5, x6, x7
EXIT:
```

Here is second possible answer.

```
bge x29, x28, EXIT
sub x5, x6, x7
EXIT:
```

Following fetches zero marks since there is no bgt instruction in RISC-V. Also, below code does not take care of equality of i and j.

```
bgt x29, x28, EXIT
sub x5, x6, x7
EXIT:
```

7. (4 points) For the following C statement, write the corresponding RISC-V assembly code. Assume that the variables i, and j are assigned to registers x28, and x29, respectively. Assume that the base address of the arrays A is in register x10 and sum is in x11. No need to store anything in stack.

```
for(int i=0; i<j; i++)
    sum = sum+A[i];
```

Here is one possible answer.

```
    addi x28, x0, 0
LOOP: bge x28, x29, EXIT
    slli x12, x28, 3
    add x12, x12, x10
    ld x7, 0(x12)
    add x11, x11, x7
    addi x28, x28, 1
    beq x0, x0, LOOP
EXIT:
```

If someone has checked the branch condition towards the end of their code, then it is a wrong code, because in for loop in C++, the condition is checked first. We will deduct 2 marks, assuming rest of the code is correct. Here is the code that fetches maximum 2 marks.

```
add x28,x0,x0
Loop: slli x30,x28,3
add x30,x30,x10
ld x31,0(x30)
add x11,x11,x31
addi x28,x28,1
blt x28,x29,LOOP
beq x0,x0,EXIT
EXIT:
```

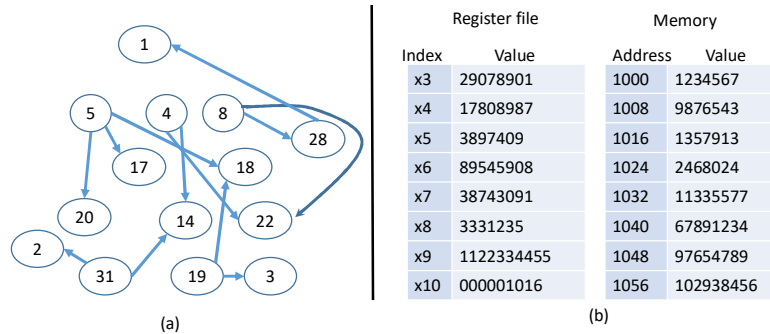


Figure 1: (a) VLIW Scheduling figure (b) RF-Memory data figure

8. (3 points) For Figure 1(a), find the number of packets if each packet has 3 instructions (1 mark). Also show which instructions are part of each packet (2 mark).

5 packets. Here is one possible answer. Those who have written other answers should send those answers to me.

4 5 8
17 20 19
18 22 28
31 1 3
2 14

Another answer:

4 5 8
17 19 20
3 18 28
22 31 1
2 14

9. (2 points) Consider below instructions that are executed when the state of register file (RF) and memory is as given in Figure 1(b). All values in this figure are in decimal. Show only the changed values in RF and memory after these instructions are executed.

ld x6, 16(x10)
sd x5, 32(x10)

x6 becomes 11335577

Mem address 1048 is changed to 3897409